

Evaluating Iceberg-Style Optimistic Concurrency Control for OLTP Workloads

Arik Gershman
University of Maryland
arikger@umd.edu

Sathya Gnanakumar
University of Maryland
sgnanaku@umd.edu

Vishnu Sreekanth
University of Maryland
vishnus@umd.edu

Daniel Kiely
University of Maryland
dmkiely@umd.edu

Mukund Shankar
University of Maryland
mukunds@umd.edu

Abstract

Modern data systems increasingly blur the distinction between on-line transaction processing (OLTP), which emphasizes concurrent transactional updates, and online analytical processing (OLAP), which focuses on large-scale read queries. Apache Iceberg is an open table format that uses optimistic concurrency control (OCC) in conjunction with immutable snapshots, making it a useful point of comparison with PostgreSQL-style multi-version concurrency control (MVCC). In this work, we investigate the viability and performance of Iceberg-style concurrency control when applied to a traditional transactional workload. We mimic Iceberg’s architecture and commit protocol within an existing in-memory database implementation to directly compare Iceberg-style concurrency control with Two-Phase Locking (2PL), Parallel OCC, and MVCC across workloads varying in contention level, transaction duration, and read/write mix. We find that Iceberg achieves throughput comparable to 2PL and OCC for read-only workloads under both low and high contention, and relatively low throughput for low-contention write-heavy workloads. However, Iceberg achieves relatively high throughput in high-contention write-heavy workloads and performs among the best in high-contention mixed workloads alongside MVCC. These results suggest that an Iceberg-inspired concurrency-control design can be competitive with mainstream transactional schemes under some workload conditions, particularly when workloads contain a substantial read component.¹

Keywords

Apache Iceberg, Concurrency Control, Optimistic Concurrency Control (OCC), Multi-Version Concurrency Control (MVCC), Online Transaction Processing (OLTP), Online Analytical Processing (OLAP), Lakehouse Architecture, Database Systems

1 Introduction

The evolution of data architecture has increasingly blurred the lines between traditional relational database management systems (RDBMS) designed for online transaction processing (OLTP) and data lakes designed for online analytical processing (OLAP). OLTP databases prioritize low-latency, high-concurrency transactional updates, while OLAP systems focus on large-scale, read-heavy analytical queries. Historically, choosing between a transactional database and an analytical data store was a defining crossroad for developers. Today, modern architectures allow for the seamless

blending of the two [4, 6, 7]. At the forefront of this convergence is Apache Iceberg [3], an open table format that brings SQL-like abstractions and ACID transaction guarantees to data stored in distributed file systems. While Iceberg is fundamentally optimized for analytical workloads, it supports point-in-time snapshots and row-level UPDATE and DELETE commands.

Unlike traditional OLTP systems that rely on fine-grained, row-level concurrency control mechanisms such as Two-Phase Locking (2PL) or traditional Multi-Version Concurrency Control (MVCC) [1], Iceberg implements isolation through a coarse-grained, Optimistic Concurrency Control (OCC) approach [5, 8, 10]. Updates in Iceberg are not made in place; instead, transactions write new data files optimistically and rely on an atomic swap of a metadata pointer to commit changes. If multiple transactions attempt to modify the table concurrently, Iceberg validates whether the underlying files overlap, aborting and retrying if a conflict is detected [3].

This architecture comes with tradeoffs: Iceberg allows unrestricted reads through snapshots, similar to MVCC, at the cost of expensive writes in the presence of conflicts. This approach scales exceptionally well for massive analytical datasets, but its suitability for highly concurrent transactional workloads remains unclear. Iceberg writes further suffer from coarse-grained conflict detection, as conflict detection is handled at the file level rather than the row level, which can introduce false conflicts and thus elevated abort rates under contention.

In this paper, we investigate the viability and performance of Iceberg-style concurrency control when applied to a traditional transactional workload. Specifically, we introduce logical *data files* and *manifests* into a key-value storage engine to simulate Iceberg’s metadata tree. We then evaluate this Iceberg-inspired concurrency control scheme against several classical implementations, including conservative 2PL, OCC with parallel validation, and a simplified PostgreSQL-style MVCC protocol. Our study addresses the following research questions:

- (1) How does Iceberg’s concurrency control model compare to traditional concurrency control mechanisms on OLTP workloads?
- (2) How do workload characteristics (transaction duration, contention level, and read/write ratio) affect the throughput and abort behavior of Iceberg-style concurrency control?
- (3) What are the fundamental tradeoffs when adapting Iceberg’s concurrency model to an in-memory transactional setting?

¹Our code is available at https://github.com/MukundShankar-dev/624_final.

2 Related Work

Several previous works have contrasted Iceberg against other systems or analyzed its performance in a variety of different contexts, though none have directly compared or contrasted it with classic OCC or PostgreSQL-style MVCC.

Aréjula Aísa [5] draws a comparison between Iceberg and Delta Lake, a storage layer with DBMS-like functionality that guarantees ACID transactions. They underscore the similarities and differences in the concurrency control policies that both leverage: while both Iceberg and Delta Lake use OCC, they guarantee snapshot isolation differently, with Iceberg utilizing atomic metadata swaps and Delta Lake employing transaction log replay for snapshot creation. By relaxing Delta Lake’s conflict detection and thereby optimizing its OCC implementation, they find that Delta Lake’s throughput can be significantly improved. While Aréjula Aísa [5] provides a qualitative contrast between Iceberg and Delta Lake, their empirical focus remains confined to Delta Lake’s performance. They contrast Iceberg’s architectural model as a conceptual baseline for a Delta-specific optimization rather than evaluating Iceberg’s performance directly.

Mishra [9] conducts a qualitative analysis of Iceberg and Databricks’ data-engineering platform, as both are highly prevalent in data analytics. They highlight how Iceberg’s snapshot isolation supports concurrent reads and writes, similar to MVCC, which motivates our comparison of Iceberg-style concurrency control to OCC and MVCC due to how it combines aspects of both. They further reinforce this through pointing out how Iceberg maintains its speed in workloads with concurrent reads and writes when comparing it against Databricks, despite their conclusion that Databricks is overall more performant and streamlined. Our work differs from Mishra [9]’s in that their comparison is largely qualitative and compares Iceberg against a Delta Lake-based platform rather than one with sole OCC or MVCC.

Saha [10] primarily focuses on Iceberg and delves into its architecture, features, and performance. They emphasize how Iceberg’s support of concurrent reads and writes maximizes overall throughput and the efficacy of its hierarchical file-based architecture in supporting low-latency querying. They also note that Iceberg has two distinct strategies (copy-on-write and merge-on-read) to deal with write-heavy workloads, highlighting a structural attempt to mitigate the write-amplification and contention issues that traditionally challenge both OCC and MVCC architectures. This overall analysis is qualitative, however, and does not directly compare the performances of Iceberg, OCC, and MVCC against a variety of workloads.

All together, these works highlight Iceberg’s support for snapshot-based concurrent access, but they do not provide a controlled comparison between Iceberg-style concurrency and classical OLTP concurrency control schemes. Our work addresses this gap by evaluating an Iceberg-inspired design against 2PL, OCC, and MVCC within a common in-memory system.

3 System Architecture

We implemented an Iceberg-inspired concurrency control scheme within an existing transactional key-value store that supports serial execution, two locking-based protocols, optimistic concurrency

control (OCC), parallel OCC, and PostgreSQL-style MVCC. The goal is to preserve the concurrency structure of Apache Iceberg and perform controlled comparison against traditional OLTP mechanisms. Apache Iceberg supports two commit strategies depending on the catalog backend: atomic rename for file-system-based deployments, and a compare-and-swap against a metastore for catalog-based deployments; our implementation adopts the latter, using an atomic CAS on an in-memory metadata pointer [3].

3.1 Overview

The baseline system stores records as key-value pairs and executes transactions through a `TxnProcessor` that schedules requests under different concurrency control modes. Traditional modes operate over mutable row-oriented storage or per-key version chains. In contrast, our Iceberg-inspired design represents database state through immutable metadata objects and commits updates by atomically replacing a root metadata pointer.

The architecture consists of three main components:

- (1) The class `IcebergStorage`, which stores the visible database state as immutable metadata snapshots.
- (2) An Iceberg execution path in `TxnProcessor`, which executes each transaction against a pinned snapshot.
- (3) A CAS-based commit protocol that attempts to install a newly constructed snapshot and retries on conflict.

This design mirrors the core idea behind Iceberg: writers never update existing state in place. Instead, they generate new immutable metadata artifacts and attempt to publish them as the latest table snapshot.

3.2 Metadata Hierarchy

We model table state using a three-level metadata hierarchy.

3.2.1 DataFile. A `DataFile` represents a single logical key-value write. Each object stores a unique identifier, a key, and its corresponding value. Although real Iceberg data files contain many rows, we use one logical write per `DataFile` to isolate the metadata-driven concurrency behavior. We discuss the limitations of this choice in Section 6.1.

3.2.2 ManifestFile. A `ManifestFile` groups the writes produced by one transaction. When a transaction commits, all new `DataFiles` are packaged into a newly created manifest. In our implementation, each manifest also stores `lower_bound` and `upper_bound` values over the keys it contains. These bounds are used to optimize read operations, as detailed in Section 3.4.

3.2.3 MetadataFile. A `MetadataFile` represents a complete snapshot of visible table state at one logical time. Each snapshot stores a unique snapshot id and a list of pointers to the manifests that are visible in that snapshot. New snapshots reuse previously visible `ManifestFiles` and append one new `ManifestFile` for the committing transaction.

Together, these structures form an immutable metadata tree: `MetadataFile` -> `ManifestFiles` -> `DataFile`.

3.3 Storage Layer

The storage layer introduces an `IcebergStorage` class that extends the baseline mutable key-value access model with an immutable snapshot-oriented design for the Iceberg mode. At the core of the storage layer is an atomic pointer, `current_metadata_`, which references the most recent visible `MetadataFile`. This pointer serves as the logical root of the database and identifies the snapshot currently visible to the system. In addition, `IcebergStorage` maintains counters for snapshot, manifest, and data-file identifiers, as well as ownership of all allocated metadata objects. Together, these components provide hierarchical access to immutable metadata through the current snapshot pointer.

During initialization, the storage layer constructs the first database snapshot by creating an initial `ManifestFile` containing all benchmark keys, each represented by a corresponding `DataFile` initialized to value zero. A `MetadataFile` is then created to reference this initial manifest, and the atomic root pointer is set to this snapshot. As a result, the database begins execution with a fully materialized immutable metadata state rather than a mutable row store.

Metadata snapshots serve as the source of truth for database state. Rather than resolving reads from in-place records, reads are satisfied by traversing the metadata hierarchy rooted at the current snapshot to locate the most recent visible `DataFile` associated with a requested key.

3.4 Read Path Optimization

To efficiently query the metadata hierarchy and prevent read operations from becoming a bottleneck, we implemented a read path optimization leveraging binary search and Iceberg-inspired min/max filtering [3]. In a naive implementation, resolving a read request requires a linear scan ($O(N)$) across all data files within the visible manifests, stored as a list in the `MetadataFile`.

During a read operation, we implemented the tree traversal function to first evaluate the target key against the manifest's `lower_bound` and `upper_bound`. If the target key falls outside these bounds, the manifest is pruned in $O(1)$ time without examining its contents. If the manifest passes this bounds check, the system leverages the sorted order of the underlying data files to perform a binary search ($O(\log N)$) rather than a linear scan.

3.5 Transaction Execution

Transactions in our Iceberg implementation are first passed to the scheduler, which runs on a dedicated thread. After being passed to an execution thread, execution follows three phases: snapshot pinning, the read phase, and the execution logic.

3.5.1 Scheduler. The Iceberg scheduler dispatches each incoming transaction as an independent task to a thread pool by calling `IcebergExecuteTxn()`, similar to the parallel OCC scheduler. Since each transaction reads from immutable snapshots and buffers writes locally until commit time, there is no synchronization required between concurrently executing transactions.

3.5.2 Snapshot Pinning. When a transaction begins, it calls `GetCurrentMetadata()` to record a pointer to the current `MetadataFile`. This pointer serves as the transaction's consistent view of

the database for the duration of its execution. Notably, we require no locking for reads because metadata snapshots are immutable.

3.5.3 Read Phase. For each key in the transaction's read and write sets, the executor traverses the `ManifestFiles` from the pinned root and reads the corresponding `DataFile` while applying the read path optimizations described in Section 3.4.

3.5.4 Execution. Similar to OCC, after the read phase the transaction then executes writes and stores them locally in `txn->writes_`. If the transaction self-aborts, it is immediately aborted and pushed to the results queue. Otherwise, we proceed with the CAS-based commit protocol.

3.6 CAS-Based Commit Protocol

To handle conflicts, our Iceberg implementation leverages a Compare-and-Swap (CAS) commit protocol which combines aspects of OCC and MVCC together through making use of `DataFiles`, `ManifestFiles`, and `MetadataFiles` files in different ways. This portion of the implementation lies in the latter part of `IcebergExecuteTxn()` inside the `TxnProcessor` class.

3.6.1 Read-Only Transactions. Once a transaction has executed, the protocol first checks if the transaction is read-only, in which case it can follow a fast path and immediately commit due to the snapshot isolation afforded by the immutable pinned `MetadataFile`. This is the heart of the MVCC-like element of our implementation: the use of snapshots guarantees that reads are never blocked and remain unaffected by concurrent transactions.

3.6.2 Write Transactions. If the transaction is not read-only, then a new `ManifestFile`, which we name `new_manifest`, is created, containing all of the transaction's writes. Then, for each modified record, a new `DataFile` with a unique identifier containing the write is created and appended to `new_manifest`. Once this is complete, we store the pinned `MetadataFile` from the beginning of the transaction, naming it `expected`. We then initialize two boolean flags, `committed` and `conflict`, to keep track of whether the transaction has committed and whether a conflict has been detected, respectively. This is where the CAS loop begins. The CAS loop runs while both `committed` and `conflict` are false. At the start, the CAS is immediately attempted, starting off with the comparison between the system's current `MetadataFile` pointer and `expected`.

CAS Success. If the `MetadataFile` pointer is pointing to `expected`, then no other transaction has committed since the beginning of this transaction, and the comparison is successful: the `MetadataFile` pointer can now swap out `expected` and point to a new `MetadataFile` containing the union of all previous manifests and `new_manifest`. Now, `committed` gets set to true, breaking out of the CAS loop, after which the transaction can safely be committed. This is the optimistic case which Iceberg's OCC is based upon.

CAS Failure. If the metadata pointer is not pointing to `expected`, then another transaction has committed since the beginning of this transaction, and the metadata pointer has advanced. This introduces potential conflicts which must be accounted for through validation. `expected` is then updated to point to the current `MetadataFile`

referenced by the metadata pointer. Then, all of the newly committed ManifestFiles added by the other transaction are checked: if there is any overlap between this transaction’s readset or writeset and the keys present in the DataFiles included within the new ManifestFiles, conflict is set to true and the CAS loop is terminated, with the transaction duly being aborted and re-queued into the TxnProcessor. If there is no overlap detected throughout all of the new ManifestFiles, then no flags are updated and the CAS loop simply continues for another iteration, in order to account for the possibility of any more concurrent transactions occurring during this validation process. This is made possible because expected was updated to what the metadata pointer was pointing to, accounting for this new version of the MetadataFile and noting it as expected for the next iteration since it has been successfully validated.

Like real Iceberg, our implementation provides serializable isolation: the CAS-based commit protocol ensures any set of committed transactions is equivalent to some serial execution, aborting transactions with conflicting write sets while allowing non-conflicting transactions to append independent manifests. Reads are never blocked, as the immutable metadata tree guarantees a pinned snapshot remains valid for the duration of any transaction regardless of concurrent commits.

3.7 Comparison to Baseline Protocols

There are a few differences between the Iceberg-inspired architecture and the baseline protocols. First, our system separates visibility from the mutable storage state. Instead of updating rows in place, the system exposes the database state through immutable metadata snapshots. Second, validation occurs at commit time via the root pointer comparison and manifest inspection rather than locking or centralized serial validation. This makes the design closer to metadata-driven lakehouse systems than to traditional OLTP concurrency control. By designing each DataFile to contain one key-value write, we are able to directly evaluate the impact of Iceberg’s immutable metadata snapshots and CAS-based commit protocol.

4 Experiments

To evaluate the relative performance of Iceberg-style OCC for OLTP workloads, we benchmarked our custom ICEBERG implementation against Conservative 2PL (LOCKING), Parallel Validation OCC (P_OCC), and PostgreSQL-Style Multi-Version Concurrency Control (MVCC). We omit serial OCC due to its similarity to P_OCC across all benchmarks.

All experiments were executed using a static thread pool of 8 worker threads managing a fixed pool of 100 active transactions. To simulate varying computational overheads in transactional logic, we configured transaction durations across the workloads to 0.1ms, 1.0ms, and 10.0ms. We modeled contention levels by varying the database key space, with high-contention workloads operating on a narrow range of 100 keys, and low-contention workloads operating on a broader range of 1,000,000 keys. For read-only workloads, transactions access either 5 or 30 records. For read-write workloads, transactions write either 5 or 10 records. Our mixed workload combines 30-record reads and 10-record writes. Our primary evaluation

metrics are transaction throughput (Transactions Per Second, or TPS) and the transaction retry rate (percentage of aborted transactions).

4.1 Read-Only Throughput Analysis

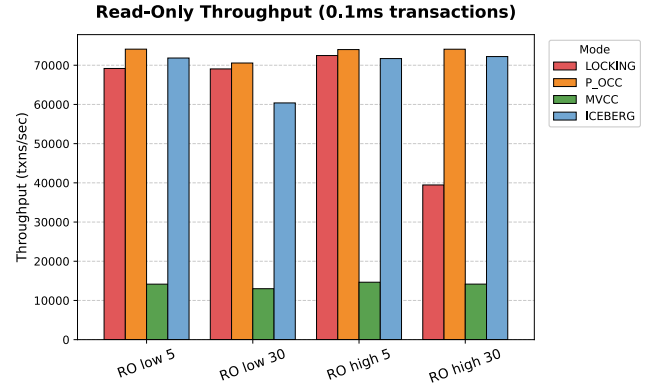


Figure 1: Throughput across read-only workloads at a 0.1ms transaction duration.

In our evaluation of strictly read-only workloads, ICEBERG, P_OCC, and LOCKING achieved near-identical throughput, as shown in Figure 1. Because read operations do not mutate the database state, they inherently generate no transaction conflicts. Consequently, none of the optimistic validation penalties, locking overheads, or manifest-generation costs are incurred, allowing these systems to scale effectively based purely on transaction duration. The notable exception is MVCC, which incurs a fixed per-transaction cost for snapshot creation. This overhead, at a 0.1ms duration, represents a substantial fraction of total execution time and produces a predictable throughput bottleneck. Notably, ICEBERG does not suffer from this bottleneck because it doesn’t have to take an expensive snapshot at the start of each transaction; rather, it just pins the current MetadataFile pointer. This is possible because ICEBERG writes are simply a union of a new ManifestFile with the existing ManifestFiles, rather than a modification of state in place. Since the metadata tree is immutable, the pinned pointer is guaranteed to remain a valid, consistent view of the database for the duration of the transaction, with no risk of concurrent writes invalidating it.

Interestingly, we observed that ICEBERG and the other top performing protocols actually demonstrate slightly *higher* throughput under high contention than low contention. While counterintuitive from a concurrency perspective, we hypothesize that this is due to hardware-level cache locality. In our read-only high-contention context, all threads are repeatedly reading a small subset of database keys. This drastic reduction in the working set size compared to low contention allows the targeted records, and Iceberg’s associated metadata tree structures, to fully stay in the CPU’s cache. Although main memory fetches are fast, the CPU cache allows for even faster data retrieval. Therefore, this slight performance bump is likely not caused by a concurrency control feature, but rather by minimized cache misses and reduced main memory fetches.

4.2 The Impact of Contention on Write-Heavy Throughput

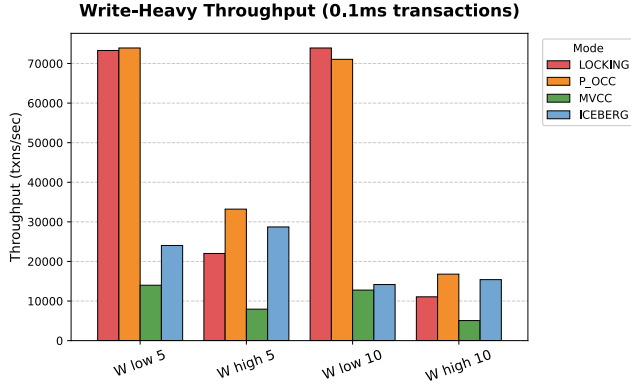


Figure 2: Throughput across write-heavy workloads at a 0.1ms transaction duration.

Our experiments revealed an interesting performance degradation for our ICEBERG implementation under low-contention write-heavy scenarios for very short transactions (0.1ms), as shown in Figure 2. In low-contention environments, optimistic transactions rarely abort, meaning new manifests are constantly being created and appended to a copied manifest list, especially when transactions execute quickly. Each transaction updates 5 to 10 random records, with keys randomly dispersed across the 1,000,000-key range. Due to this randomness, the lower and upper bounds of each generated manifest are likely to span a vast key range, creating heavy overlap of ranges across manifests. When a read operation traverses the metadata, the $O(1)$ bounds check fails to prune most manifests, as the target key falls within the range of nearly every manifest. Therefore, the system is forced to execute $O(M \log N)$ binary searches across a surging manifest list, where M is the number of manifests and N is the number of data files per manifest. As low-contention write transactions commit rapidly, M grows continuously, compounding the traversal cost and degrading read throughput. For fast write transactions, this becomes a performance bottleneck, degrading overall throughput. However, for slower transactions (1.0ms and 10.0ms), the execution delay far exceeds the traversal cost, and the lower commit frequency keeps the manifest list short enough, allowing low-contention throughput to behave as expected. Further, when more records are written per transaction, we get more overlap in the manifest bounds, resulting in more binary searches being performed and thus worse throughput.

Since optimistic transactions in high-contention environments abort more frequently, the manifest list grows significantly slower. The $O(M \log N)$ traversal benefits from a shorter list because the $O(1)$ pruning remains effectively irrelevant in this environment.² This explains why write-heavy throughput using only binary search was comparable to throughput using both min/max filtering and

²The main difference here is that the database key range is smaller, but transactions' ranges are still highly likely to contain a significant percentage of the total key range.

binary search.³ This is an inherent problem with distributed lake-house architectures; it can be mitigated through background data compaction, which may be explored in future work.

4.3 Mixed Workloads

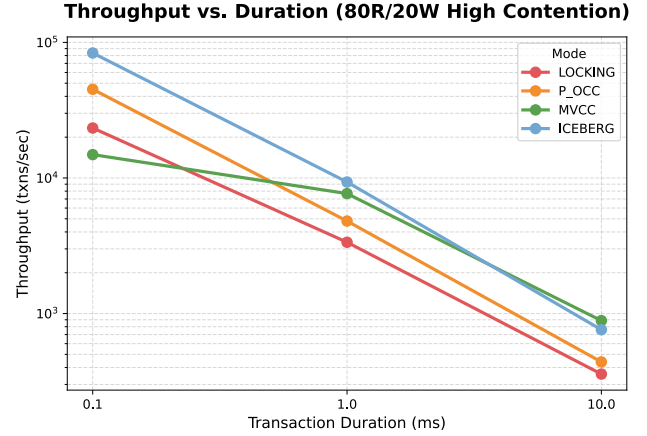


Figure 3: Throughput across varying transaction durations for the 80R/20W high-contention workload.

To evaluate mixed read-write transactional conditions, we tested a workload of 80% reads and 20% writes under high contention. Under these conditions, we would expect MVCC to perform well due to unrestricted reads, while P_OCC will suffer from frequent aborts and LOCKING will suffer from blocking. Figure 3 reveals an interesting performance dynamic between ICEBERG and MVCC across transaction durations. At 0.1ms, ICEBERG dramatically outperforms MVCC, achieving nearly 84,000 TPS compared to MVCC's 15,000, also significantly outperforming P_OCC and LOCKING. This gap shrinks but still holds at the 1.0ms duration. This should not be surprising because ICEBERG achieves the same benefits of MVCC: reads are unrestricted, even in the presence of high-contention writes. However, ICEBERG has negligible overhead for reads due to the immutable metadata tree, so it greatly outperforms MVCC under short-duration transactions. Short transactions minimize the window during which a concurrent commit can invalidate an optimistic transaction, so ICEBERG commits succeed at high rates. MVCC, by contrast, incurs structural overhead from taking snapshots before every read and write. With short transactions, this overhead bottlenecks MVCC and prevents it from outperforming ICEBERG.

However, MVCC slightly outperforms ICEBERG when transactions are 10.0ms long. With longer transactions, conflicts are more likely, resulting in a lot of wasted work and thus decreased throughput. With long transactions, MVCC's overhead is no longer a bottleneck, and we see that MVCC slightly outperforms ICEBERG, although the difference is minimal.

P_OCC performs worse than ICEBERG at all durations and follows the same declining throughput as transaction durations increase,

³The cheap $O(1)$ pruning is still a feature worth keeping, despite its rarity in our testing environment. Under workloads with more structurally distributed keys, this optimization should prove highly effective.

with LOCKING also following a similar pattern. Notably, MVCC is the only scheme in which its *relative* throughput improves as transaction duration increases due to the startup overhead mentioned above.

4.4 Abort and Retry Rates

The retry percentage data illustrates the core weakness of OCC-based systems, including ICEBERG, when applied to OLTP workloads.

For read-only workloads, the retry rate remains at 0%, as expected. However, in the high-contention write-heavy workload, the retry rate skyrockets across all optimistic variations, as shown in Figure 4. The retry percentage hovers between 55% (5 records) and 75% (10 records) at all transaction durations. Notably, ICEBERG and P_OCC demonstrate nearly identical retry rates and patterns across all durations. This is an expected result, since both resolve conflicts entirely at commit time and are equally blind to contention during execution.

Because Iceberg-style concurrency resolves conflicts entirely at the commit phase, much like OCC, a transaction that fails validation must discard all uncommitted writes and restart with a fresh ID. This means that under high contention, more than half of the system’s computational cycles are wasted on discarded work. Our results demonstrate that even MVCC’s retry percentage is comparable to or worse than optimistic protocols under these workloads, indicating that even fine-grained versioning cannot mitigate a large volume of conflicting writes.

5 Discussion

Our results suggest that Iceberg-style concurrency can be competitive in OLTP workloads, but only under specific workload conditions. In read-heavy settings, immutable snapshots provide a clear advantage by allowing transactions to read without acquiring locks or aborting. After adding manifest-level pruning and binary-search-based lookup, our Iceberg-inspired implementation achieved throughput comparable to classical OCC and locking schemes on read-only workloads. This indicates that metadata-based reads can be efficient when the cost of metadata traversal is controlled.

However, the experiments also highlight the core weakness of optimistic snapshot-based concurrency under write contention. In high-contention write-heavy workloads, Iceberg’s retry behavior closely resembles OCC. This is because conflicts are detected only at commit time, so transactions may execute for their full duration before being discarded, wasting substantial computational work. This effect is magnified for longer transactions, where each abort represents a larger amount of wasted execution time. Our results therefore suggest that while Iceberg-style concurrency can reduce synchronization overhead for reads, it remains vulnerable to wasted work under frequent conflicting writes. In mixed, read-heavy workloads, Iceberg performs very well because reads are unrestricted. Further, if transactions are very short, then Iceberg outperforms MVCC in these conditions. Ultimately, a read-heavy workload with short transactions in the presence of some low contention writes is the optimal setting for Iceberg-style concurrency in main-memory databases.

One notable result from our experiments is the reduced throughput of Iceberg under low-contention write-heavy workloads with short transactions. This behavior can be explained by the structure of the metadata. Under low contention, transactions commit frequently, causing manifests to accumulate rapidly. Since each transaction writes a small number of randomly distributed keys, the resulting lower and upper bounds of many manifests overlap heavily. This reduces the effectiveness of min/max pruning optimization and forces the system to perform binary searches across a growing number of manifests. As a result, background compaction or a more structured file grouping strategy may improve performance in low-contention write-heavy workloads. This behavior indicates that Iceberg-inspired designs depend not only on the contention and transaction duration, but also on the evolving structure of its metadata. In production systems, compaction bounds the total number of ManifestFiles, and thus greatly improves the read performance [2].

6 Conclusion

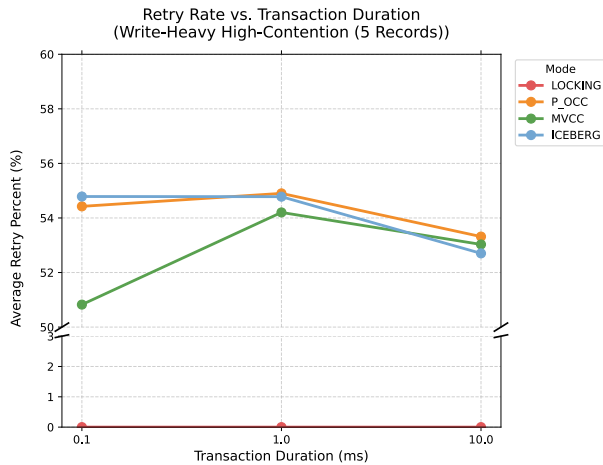
In this work, we evaluated whether an Iceberg-style concurrency control protocol can be applied to OLTP workloads. We implemented an in-memory Iceberg-style protocol using immutable DataFiles, ManifestFiles, and MetadataFiles, and then compared it against 2PL, Parallel OCC, and PostgreSQL-style MVCC by varying contention level, transaction duration, and read/write ratios. Our results show that our Iceberg-style approach is highly competitive for read-only and read-heavy workloads, where the immutable snapshots allow transactions to execute without any blocking, avoiding much of the synchronization costs associated with the other protocols. Our Iceberg-style protocol achieved strong performance (comparable to 2PL and parallel OCC) on read-only workloads and performed notably well on short, high-contention mixed workloads.

At the same time, our experiments also reveal some limitations. Since conflicts are only detected during commit time, transactions executing under our Iceberg-style protocol suffer under write-heavy contention in the same way as OCC. Specifically, the transactions may spend their full execution time doing work that is eventually aborted. We also observed that write-heavy workloads in low-contention settings show some degraded performance due to metadata management overhead, since frequent successful commits increase the number of manifests significantly, which reduces the effectiveness of min/max pruning when key ranges have significant overlap. Our results suggest that the performance of Iceberg-style concurrency strongly correlates with metadata growth and layout.

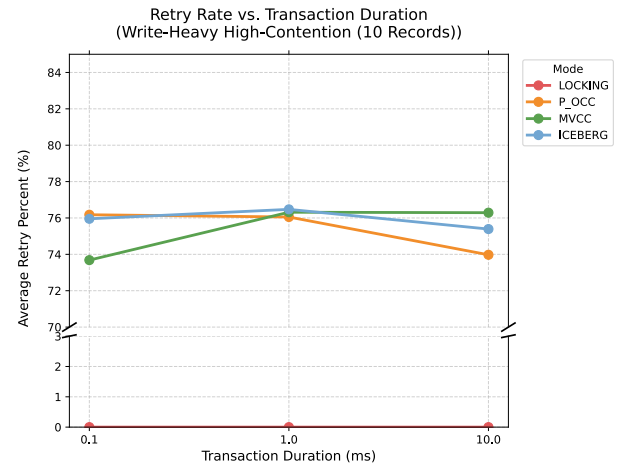
Overall, our results indicate that Iceberg-style concurrency control is not a universal replacement for traditional OLTP concurrency control protocols, but it can be a strong competitor for applications with read-heavy workloads, short transactions, or limited conflicting writes.

6.1 Future Work

Future work can expand upon our study in various ways. Firstly, while our system is inspired by Apache Iceberg, each DataFile in our simulator represents a single logical key-value write rather than a large physical file containing many rows. An implementation that instead takes the latter approach in representing DataFiles



(a) W High-Contention 5 Records



(b) W High-Contention 10 Records

Figure 4: Retry rates across transaction durations for 5-record and 10-record write-only high-contention workloads.

could potentially yield different results, especially with write-heavy workloads, due to reduced metadata management overhead and improved I/O throughput. However, this approach comes at a cost: the standard Iceberg implementation performs coarse-grained conflict detection, only checking if there are conflicts at the DataFile-level, rather than at the record level [3]. While this approach scales better, it may result in many false conflicts and wasted work. A study that measures the throughput of such an implementation would be able to verify this.

Additionally, further work could explore an implementation that leverages background data compaction to potentially prevent performance degradation for low-contention write-heavy workloads. This would verify our hypothesis that our implementation’s binary search over the rapidly growing manifest list coupled with frequent pruning causes a bottleneck in low-contention cases in which there are not many aborts.

Further, our study measures throughput and abort rates, but we do not measure transactional latency. Iceberg claims to support low latency workloads [4], and future work may investigate whether this claim holds true in an OLTP setting.

References

- [1] Anastasia Ailamaki, Silesh Krishnamurthy, Spiros Papadimitriou, Bianca Schroeder, Karl Schnaitter, and Gavin Sherry. 2011. Transaction Management in PostgreSQL (Section 27.4). In *Database System Concepts* (6th ed.), Abraham Silberschatz, Henry F. Korth, and S. Sudarshan (Eds.). McGraw-Hill, New York, Chapter 27, 1137–1146.
- [2] Amazon Web Services. 2024. Maintaining Tables by Using Compaction. <https://docs.aws.amazon.com/prescriptive-guidance/latest/apache-iceberg-on-aws/best-practices-compaction.html>. Accessed: 2026-05-7.
- [3] Apache Software Foundation. 2026. *Apache Iceberg Table Spec*. Retrieved May 1, 2026 from <https://iceberg.apache.org/spec/>
- [4] Apache Software Foundation. 2026. *Apache Iceberg: The open table format for analytic datasets*. Retrieved May 1, 2026 from <https://iceberg.apache.org/>
- [5] Inigo Aréjula Aisa. 2025. Improving optimistic concurrency Control in Delta Lake: Enabling safe parallel schema evolution.
- [6] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. 2021. Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. In *Proceedings of CIDR*.
- [7] Dongxu Huang, Qi Liu, Qiu Cui, Zhuohe Fang, Xiaoyu Ma, Fei Xu, et al. 2020. TiDB: A Raft-based HTAP Database. *Proceedings of the VLDB Endowment* 13, 12 (aug 2020), 3072–3084.
- [8] Hsiang-Tsung Kung and John T. Robinson. 1981. On optimistic methods for concurrency control. *ACM Transactions on Database Systems (TODS)* 6, 2 (jun 1981), 213–226. <https://www.seas.upenn.edu/~zives/cis650/papers/opt-cc.pdf>
- [9] Sarbaree Mishra. 2022. Comparing Apache Iceberg and Databricks in building data lakes and mesh architectures. *International Journal of AI, BigData, Computational and Management Studies* 3, 4 (2022), 37–48.
- [10] Dipankar Saha. 2024. Disruptor in Data Engineering-Comprehensive Review of Apache Iceberg. *SSRN Electronic Journal* (2024).